ANNOUNCEMENT

- Checkpoint II presentations will be on 11.09
  - 10-12 min presentation + 3 min Q&A
  - Track I MUST cover:
    - A research problem your team chose
    - Your scientific hypothesis, the approach, and Δ over prior work
    - Your experimental design for evaluating the hypothesis
    - Your preliminary results [very important]
    - Your next steps to complete the research at the final presentation

ANNOUNCEMENT – CONT'D

- Checkpoint II presentations will be on 11.09
  - 10-12 min presentation + 2-3 min Q&A
  - Track 2 MUST cover:
    - The method of your team chose
      - Overview (1 slide)
      - Threat model (1 slide): attacker, defender and what counts as a defense success
      - Method (1-2 slides): input and output; as-is (w/o defense) vs. to-be (with defense)
    - Reproduction target
      - The paper's results: numbers, metrics, models and datasets
      - Your success criterion: how you will judge the reproduction?
    - Preliminary results [required]
      - Your results and comparison to the paper's result
    - Next steps: what remains until the final presentation and who does it

CS 578: Cyber-security

Part III: Isolation

Sanghyun Hong
sanghyun.hong@oregonstate.edu





RECAP – WHERE WE ARE

- Part I – Network/Internet security
  - The attacker is *outside* your machine
- Part II – Computer systems security
  - The attacker is *inside* your machine
  - Memory safety: *how* the attacker gets in
  - OS security: *decide what* the attacker may do (access control)
- Part III – Isolation and (software-induced) breaks
  - The attacker is *inside your program* or is *the OS* itself

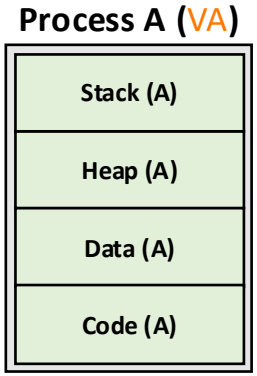
ISOLATION – WHERE ACCESS CONTROL CANNOT REACH

RECAP

- Process **isolation**
  - **Definition:** Prevent Process A from reading/writing to Process B
  - Why?
    - Security reasons (e.g., data breach, system crash, …)
    - Management reasons (e.g., easy to control, …)
  - What happens if we access the other process' memory
    - **Segmentation fault**

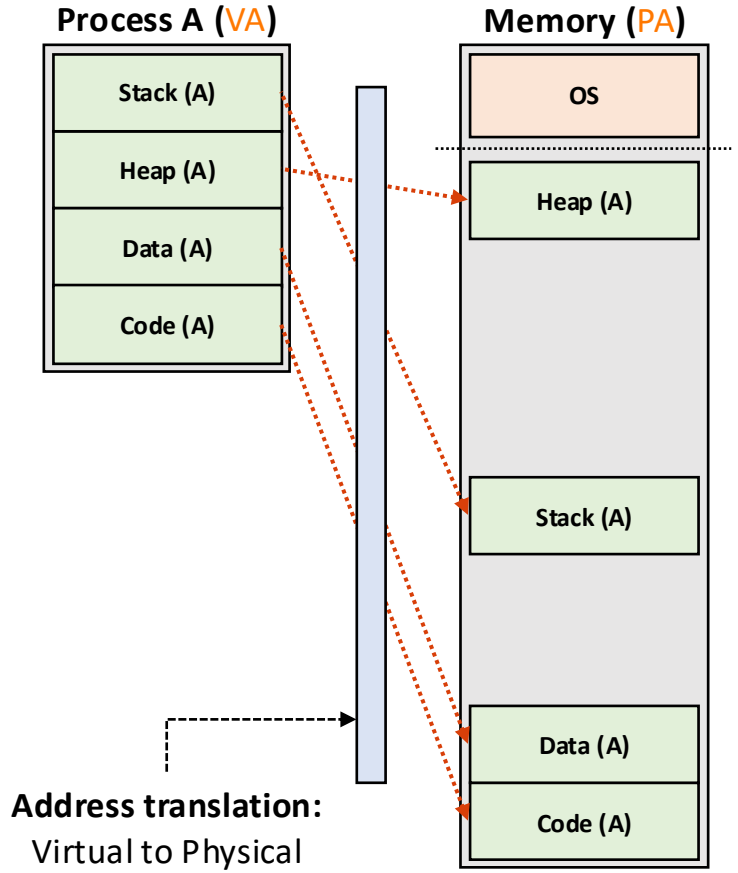
RECAP – PROCESS ISOLATION

- Process has 4 segments
  - Code segment
  - Data segment
  - Heap segment
  - Stack segment



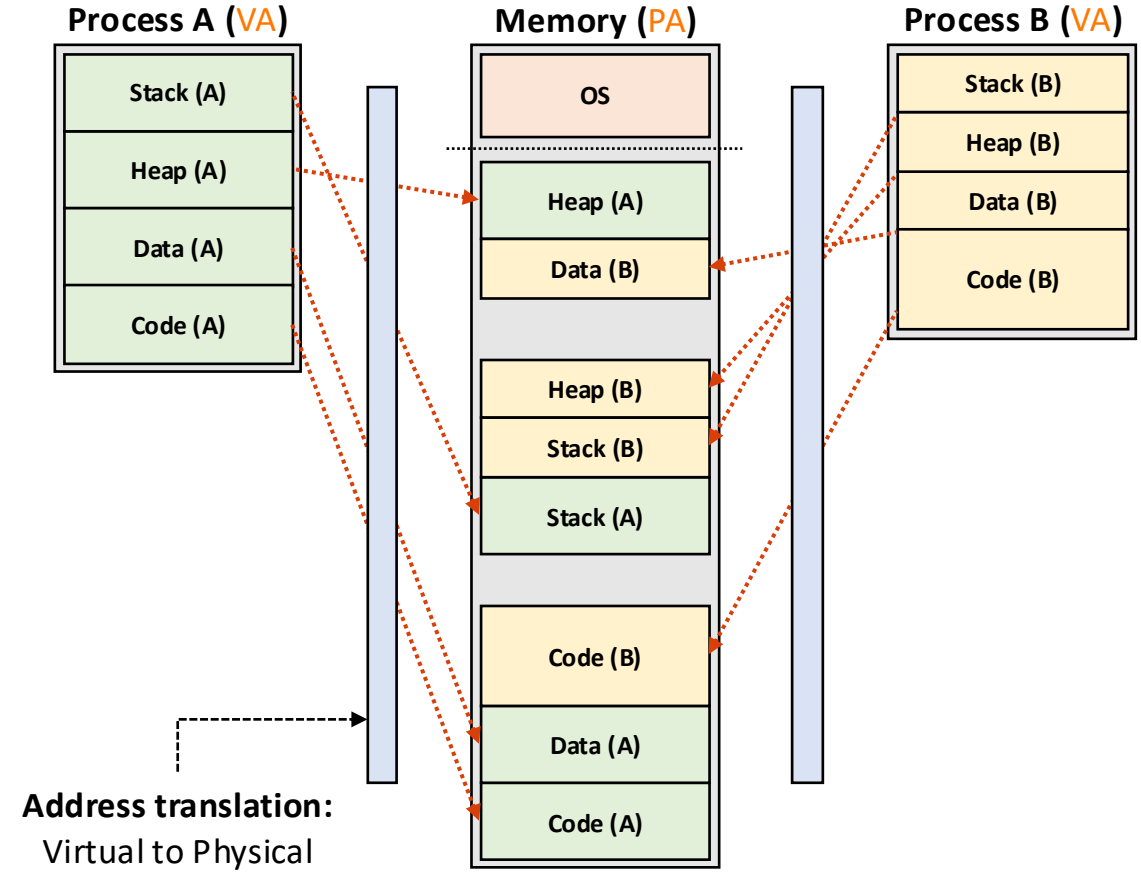
RECAP – PROCESS ISOLATION

- Process has 4 segments
  - Code segment
  - Data segment
  - Heap segment
  - Stack segment



RECAP – PROCESS ISOLATION

- Process has 4 segments
  - Code segment
  - Data segment
  - Heap segment
  - Stack segment



UNTRUSTED CODE IN YOUR PROGRAM

- Applications load code written by others
  - Database: user-defined functions (e.g., POSTGRES)
  - OS kernel: new file systems, packet filters
  - Browsers and editors: extensions and plugins

- This code runs in the same address space as the application
  - A bug (or malicious code) can corrupt the application's data
  - Users cannot tell which module caused the failure
  - Your agentic browsers nowadays[1]

[1] Lee et al., *Site Isolation is Dead: How Site Isolation is Broken in Agentic Browsers and Extensions*, IEEE S&P'26

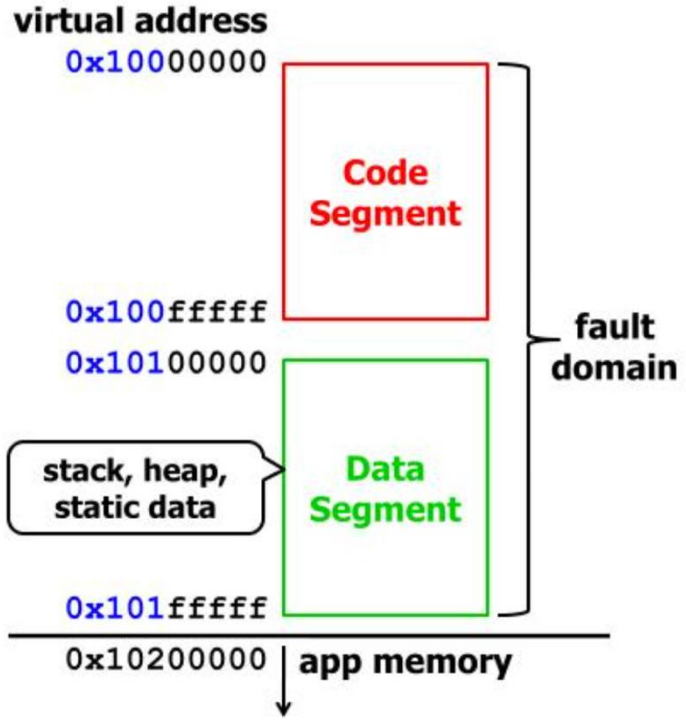
UNTRUSTED CODE IN YOUR PROGRAM – POTENTIAL SOLUTIONS

- Run each module in its own process
  - The OS' Page tables keep one process from touching another's memory
- Calling across processes is expensive (RPC)
  - Trap into the kernel, copy the arguments
  - Save and restore registers, switch address spaces (TLB flush)
  - Trap back to the user level – and repeat on return
  - 2–3 orders of magnitude slower than a procedure call
- The application and its modules call each other very often

SFI – SANDBOXING UNTRUSTED CODE INSIDE YOUR PROGRAM

SFI, SOFTWARE FAULT ISOLATION – KEY IDEA

- Challenge
  - Untrusted code runs in the application's address space
  - Separate processes isolate it, but every call needs a trap and a context switch
- Approach
  - Keep the code in the same address space, in its own fault domain
  - Fault domain: a code segment and a data segment
  - Rewrite the code so that its writes and jumps stay in the fault domain
  - Calls between domains stay cheap: no trap, no context switch



SFI – THREAT MODEL

	Adversary (untrusted module)	Defender (application + SFI)
Goal	Write or jump outside its fault domain	Keep every write and jump in the fault domain
Knowledge	Knows SFI and the memory layout	Has the module's object code
Capabilities	Runs any instruction in its fault domain, including indirect jumps	Rewrites and verifies the code, reserves registers, maps guard pages

- Assumptions
  - The OS and the hardware are trusted
  - The compiler is not trusted; the verifier is
  - RISC CPUs (MIPS, Alpha): fixed-length instructions, 32 registers
  - Out of scope: reads (confidentiality), bugs inside the module itself

SFI – UNSAFE INSTRUCTIONS

- Why look at instructions?
  - SFI must stop every write and jump that can leave the fault domain
  - Checking all of them at run time would be too slow
- Safe: the target is known before running
  - e.g., PC-relative branches, stores to a fixed address
  - Checked once, before the module runs
- Unsafe: the target is in a register
  - e.g., function pointers, returns, writes through pointers
  - SFI adds checks only before these instructions

SFI – SEGMENT MATCHING

- Check the upper bits of the target address
- Trap if they are not the segment ID
- Overhead
  - 4 more instructions
  - 4 dedicated registers
- Pros
  - Tells which instruction is faulty (debugging)

```
STORE R0, R1    ; write R1 to Mem[R0]
```



```
MOV Ra, R0      ; copy R0 into Ra
SHR Rb, Ra, Rc  ; Rb = Ra >> Rc, to get segment ID
CMP Rb, Rd      ; Rd holds correct data segment ID
BNE fault       ; wrong data segment ID
STORE Ra, R1    ; Ra in data segment, so do write
```

SFI – ADDRESS SANDBOXING

- No check; overwrite the upper bits instead
  - `addr = (addr & mask) | seg_id`
- Overhead
  - 2 more instructions
  - 5 dedicated registers
- A wrong address still stays in the fault domain
  - Cannot detect faults, but cheaper than segment matching

dedicated-reg ⇐ target-reg&and-mask-reg
Use dedicated register **and-mask-reg** *to clear segment identifier bits.*

dedicated-reg ⇐ dedicated-reg|segment-reg
Use dedicated register **segment-reg** *to set segment identifier bits.*

store instruction uses dedicated-reg

```
target      0xDEADBEEF
& mask      0x000FFFFF  -> 0x000DBEEF
| seg id    0x12300000  -> 0x123DBEEF
```

SFI – WHY DEDICATED REGISTERS?

Naive

```
r1 = r1 & mask
r1 = r1 | seg_id
store r2, [r1]   <- jump here
```

Dedicated register (dr)

```
dr = r1 & mask
dr = dr | seg_id
store r2, [dr]   <- still safe
```

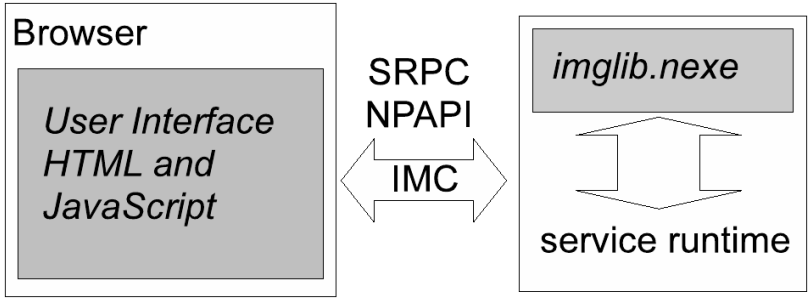
- What if the module jumps directly to the store?
  - The sandboxing instructions are skipped
- Solution: dedicated registers (dr)
  - Only the sandboxing instructions write to dr
  - dr always holds an address within the segment
  - Jumping into the middle of the sequence does not help

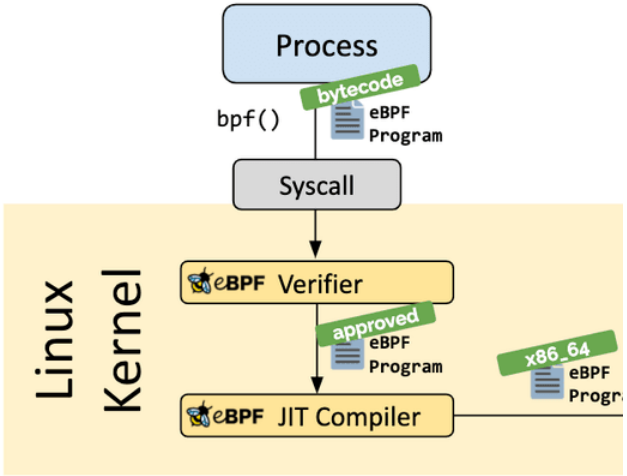
SFI – COMPILER AND VERIFIER

- Who adds the checks?
  - A modified compiler (gcc) emits the sandboxing code
- Problem: then the compiler must be trusted
  - A compiler bug, or a module built with another compiler, breaks isolation
- Solution: check the binary when it is loaded (verifier)
  - Unsafe stores and jumps use dedicated registers only
  - Dedicated registers are sandboxed after every update
  - Code that fails the check is rejected
- Only the verifier needs to be trusted (smaller TCB)

LOAD-TIME VERIFIERS TODAY

- Google Native Client (2009) (left)
  - A validator checks native x86 code before it runs in the browser
- eBPF in Linux (right)
  - The kernel verifier checks a program before it runs in the kernel



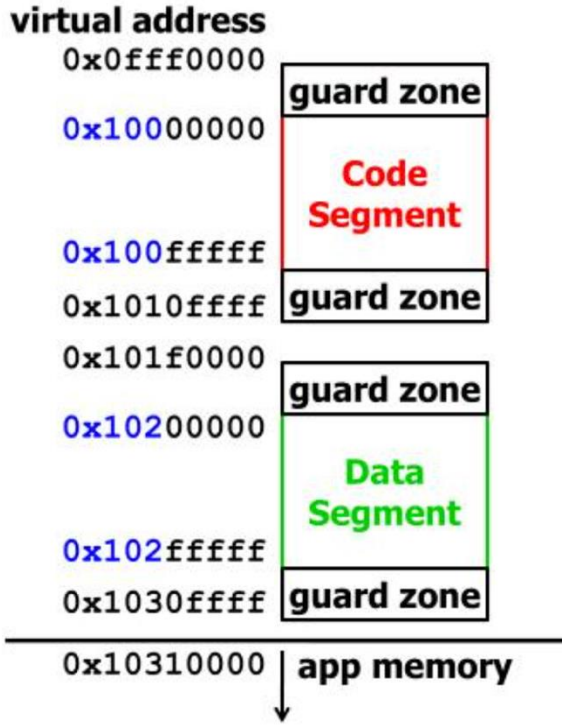


SFI – WHERE THE OVERHEAD COMES FROM

- Sandboxing adds instructions before every unsafe store and jump
  - The cost depends on how often these instructions run
- Two cases are very common
  - Stores with register+offset addressing, e.g., store value, offset(reg)
    - One more instruction to compute reg+offset before sandboxing
  - Loads and stores relative to the stack pointer
    - Local variables in almost every function
- The optimizations on the next slide target these two cases

SFI – OPTIMIZATIONS

- Guard zones
  - Unmapped pages above and below each segment
  - Sandbox reg only; reg+offset can at most reach a guard zone
  - Touching a guard zone causes a hardware fault
- Stack pointer
  - Sandbox it only when it is written
  - Small changes stay within the guard zones

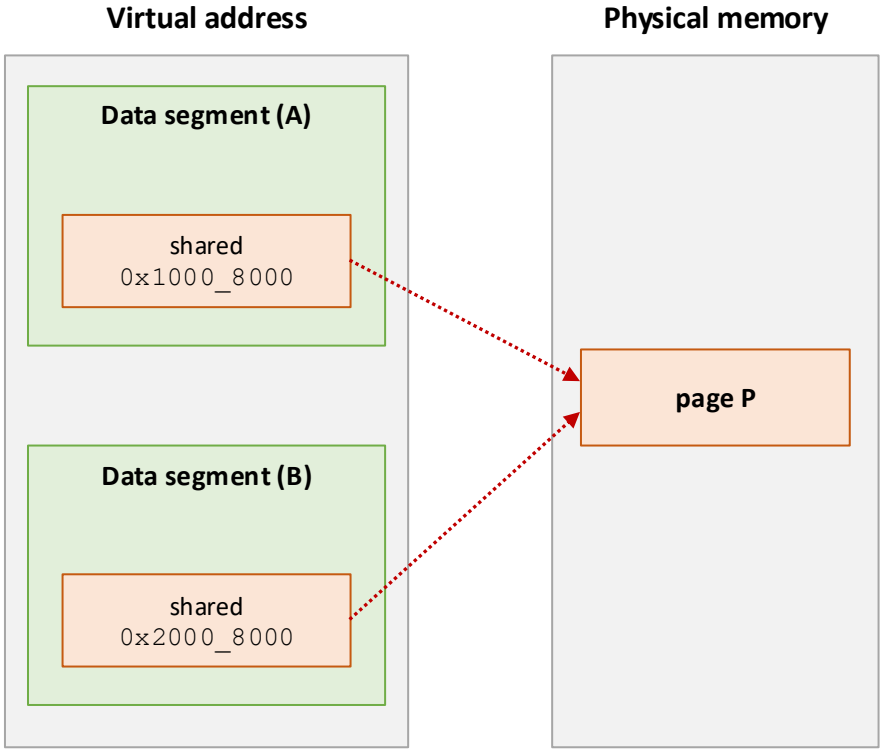


SFI – SYSTEM CALLS

- The OS only sees one process
  - It cannot tell fault domains apart
- A system call from the module has the application's full privilege
  - e.g., unlink() deletes the database file
  - e.g., mprotect() or munmap() removes the guard zones
- Solution
  - The verifier rejects system calls in the module
  - The module calls trusted code through the jump table
  - The trusted code checks the request and makes the call

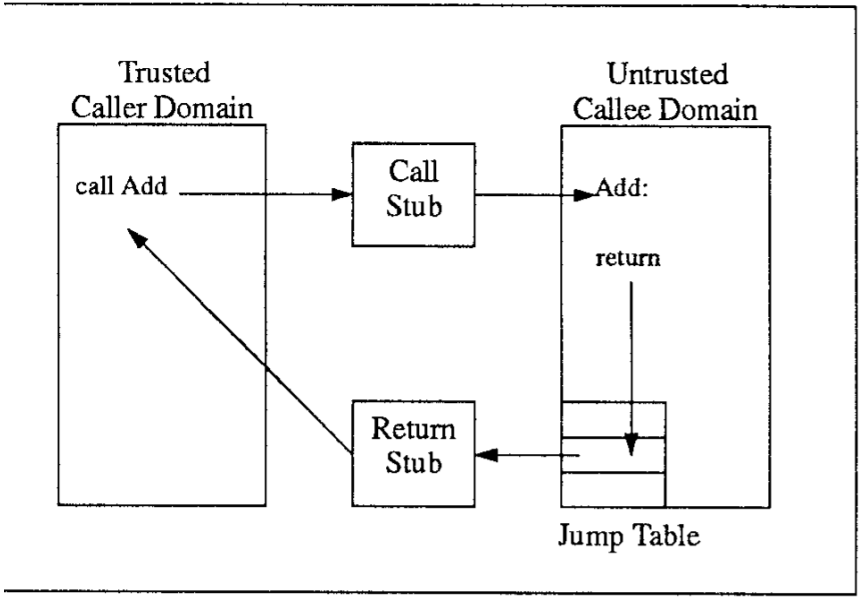
SFI – SHARING MEMORY

- A module can write only to its own data segment
- Sandboxing overwrites the upper bits
  - It cannot point into another segment
- Solution: virtual address aliasing
  - Map one physical page into both segments
  - Same offset (0x8000), different upper bits
- Copying also works, but is slow for large data



SFI – CROSSING FAULT DOMAINS

- Calls out of a fault domain go through a jump table
  - Read-only, legal entry points only
  - Also used for returns
- Call and return stubs (trusted)
  - Copy arguments, save and restore registers
  - Switch stacks, set the dedicated registers
- No kernel trap or address space switch
  - Over 10x faster than RPC between processes



SFI – PERFORMANCE OVERHEAD

- C SPEC92 and other benchmarks
- Fault isolation
  - 4.3% on average
- Sandboxing loads as well
  - 21.8% (MIPS)
  - 17.6% (Alpha)
- Reserving 5 registers
  - 0.4%

Benchmark		DEC-MIPS					DEC-ALPHA	
		Fault Isolation Overhead	Protection Overhead	Reserved Register Overhead	Instruction Count Overhead	Fault Isolation Overhead (predicted)	Fault Isolation Overhead	Protection Overhead
052.alvinn	FP	1.4%	33.4%	-0.3%	19.4%	0.2%	8.1%	35.5%
bps	FP	5.6%	15.5%	-0.1%	8.9%	5.7%	4.7%	20.3%
cholesky	FP	0.0%	22.7%	0.5%	6.5%	-1.5%	0.0%	9.3%
026.compress	INT	3.3%	13.3%	0.0%	10.9%	4.4%	-4.3%	0.0%
056.ear	FP	-1.2%	19.1%	0.2%	12.4%	2.2%	3.7%	18.3%
023.eqntott	INT	2.9%	34.4%	1.0%	2.7%	2.2%	2.3%	17.4%
008.espresso	INT	12.4%	27.0%	-1.6%	11.8%	10.5%	13.3%	33.6%
001.gcc1.35	INT	3.1%	18.7%	-9.4%	17.0%	8.9%	NA	NA
022.li	INT	5.1%	23.4%	0.3%	14.9%	11.4%	5.4%	16.2%
locus	INT	8.7%	30.4%	4.3%	10.3%	8.6%	4.3%	8.7%
mp3d	FP	10.7%	10.7%	0.0%	13.3%	8.7%	0.0%	6.7%
psgrind	INT	10.4%	19.5%	1.3%	12.1%	9.9%	8.0%	36.0%
qcd	FP	0.5%	27.0%	2.0%	8.8%	1.2%	-0.8%	12.1%
072.sc	INT	5.6%	11.2%	7.0%	8.0%	3.8%	NA	NA
tracker	INT	-0.8%	10.5%	0.4%	3.9%	2.1%	10.9%	19.9%
water	FP	0.7%	7.4%	0.3%	6.7%	1.5%	4.3%	12.3%
Average		**4.3%**	**21.8%**	**0.4%**	**10.5%**	**5.0%**	**4.3%**	**17.6%**

SFI – CROSSING COST

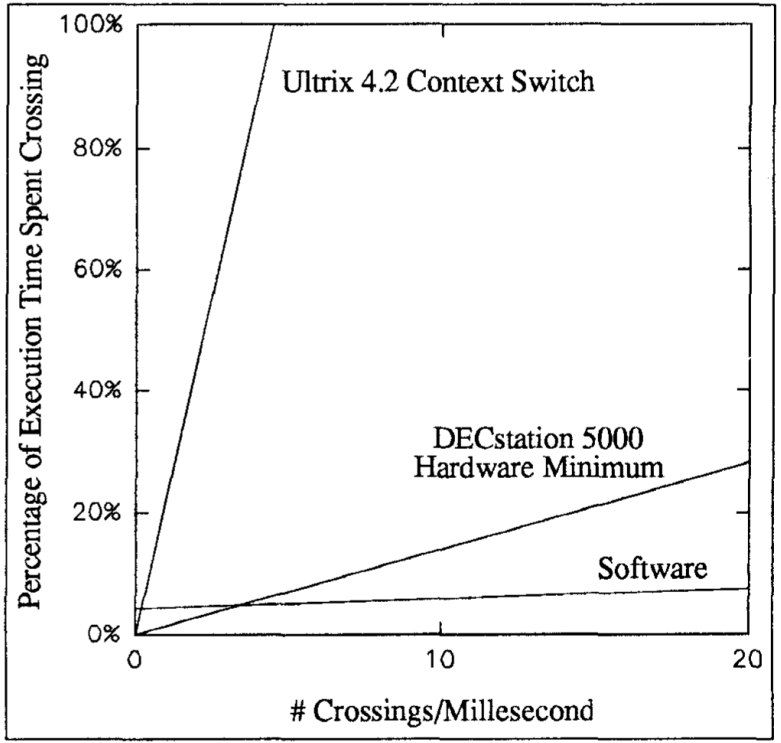
Platform	Cross Fault-Domain RPC: Caller Save Registers	Save Integer Registers	Save Integer+Float Registers	C Procedure Call	Pipes
DEC-MIPS	1.11μs	1.81μs	2.83μs	0.10μs	204.72μs
DEC-ALPHA	0.75μs	1.35μs	1.80μs	0.06μs	227.88μs

Sequoia 2000 Query	Untrusted Function Manager Overhead	Software-Enforced Fault Isolation Overhead	Number Cross-Domain Calls	DEC-MIPS-PIPE Overhead (predicted)
Query 6	1.4%	1.7%	60989	18.6%
Query 7	5.0%	1.8%	121986	38.6%
Query 8	9.0%	2.7%	121978	31.2%
Query 10	9.6%	5.7%	1427024	31.9%

- Cross-domain call
  - 1.11µs (MIPS)
  - Pipes: 204.72µs
- POSTGRES (Sequoia 2000)
  - SFI: 1.7–5.7%
  - Separate processes: 18.6–38.6% (predicted)

SFI – WHEN IS IT WORTH IT?

- SFI: a small cost on every run (about 4%)
- Hardware isolation: a large cost on every crossing
- Which one is cheaper depends on how often the domains call each other
- Figure 7 (DECstation 5000)
  - Above a few crossings per millisecond, SFI is cheaper
  - With Ultrix context switches, SFI wins almost right away
- Good fit: extensions, plugins, packet filters



SFI – PRACTICAL IMPLICATIONS

- PittSFIeld (2006): SFI for x86, aligned instruction bundles
- XFI (2006): SFI with control-flow integrity
- Google Native Client (2009): native code in the browser
- WebAssembly: bounds checks and guard pages on linear memory
  - RLBox (Firefox): sandboxing libraries with Wasm
- eBPF: a verifier for code loaded into the kernel

SFI – LIMITATIONS

- Each limitation comes from an assumption in the threat model

Assumption (threat model)	What breaks
Only writes and jumps matter	Reads are not protected; protecting loads costs 21.8% (MIPS)
RISC: fixed-length instructions, 32 registers	x86: jumps into the middle of an instruction, only 8 registers
Segments are aligned powers of two	Limits how memory can be laid out
The OS and the hardware are trusted	A malicious OS, Rowhammer, or side channels

HAVEN – SHIELDING APPLICATIONS FROM AN UNTRUSTED OS

RECAP - ISOLATION

	Who is untrusted	Who enforces	Mechanism
SFI	A module in your program	Rewritten code	Software checks
Haven	The OS and the cloud	The CPU	Hardware enclaves

- Both assume that the hardware works correctly
- Next (Wed. 10/28): Rowhammer, online (Zoom)
  - Critique: Flipping bits in memory OR Exposing the graceless degradation in DNNs

Thank You!

Sanghyun Hong

https://true-lab.ai/courses/Sec-Grad/current



